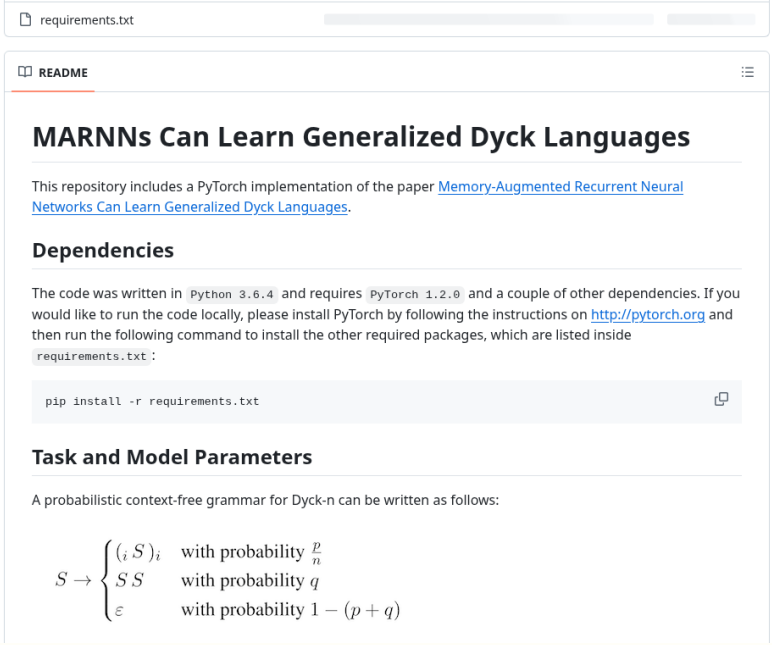
Dyck Language

括弧の入れ子構造を補完する

sasakiy84

2019 年研究の著者実装



requirements.txt

README

MARNNs Can Learn Generalized Dyck Languages

This repository includes a PyTorch implementation of the paper Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages.

Dependencies

The code was written in `Python 3.6.4` and requires `PyTorch 1.2.0` and a couple of other dependencies. If you would like to run the code locally, please install PyTorch by following the instructions on http://pytorch.org and then run the following command to install the other required packages, which are listed inside `requirements.txt`:

```
pip install -r requirements.txt
```

Task and Model Parameters

A probabilistic context-free grammar for Dyck-n can be written as follows:

$$S \to \begin{cases} (_i\, S\,)_i & \text{with probability } \frac{p}{n} \\ S\, S & \text{with probability } q \\ \varepsilon & \text{with probability } 1-(p+q) \end{cases}$$

[1] Suzgun et al. MARNNs Can Learn Generalized Dyck Languages. 著者の公式実装。 https://github.com/suzgunmirac/marnns

1. 括弧列の補完と実例
2. 正解の求め方と出力例
3. 生成方法とデータの構成
4. 研究での使われ方と入手

括弧列の補完と実例

Dyck 言語は、正しく対応する括弧の列

- Dyck-n は、 n 種類の括弧対を使い、開く括弧と閉じる括弧が正しく入れ子になる列の集合。
- ここでは丸括弧 ( ) 、角括弧 [ ] 、波括弧 { } の 3 種類を使う補完問題を紹介する。
- 入力は閉じ括弧が欠けた列。開いたままの括弧をすべて閉じる、最短の続きを答える。

[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

数だけでなく、開いた順序を記憶する

- 閉じる括弧の種類は、最後に開いたままの括弧で決まる。種類ごとの個数を数えるだけでは、閉じる順序を復元できない。
- スタックは、最後に積んだ要素を最初に取り出す記憶の仕組み。開く括弧を積み、閉じるたびに対応を照合する。
- 実例は、公開生成器で作られた異なる 13 問。入力と正解を全文で示す。

[1] Suzgun et al. (2019). Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages.
https://arxiv.org/abs/1911.03329

実例１：３つの閉じ括弧で完成する

入力（全文）／65 記号

```
( [ ( ) ( { { ( ( { { } } ) ) } { [ ( { } (
[ ] ) [ { } ] ) ] } } [ ( { ( ( [ [ [ [ [ ]
] ] ] ] ) ) } ) ] { [ ] } { ( ( ) ) } ( )
```

正解：）] ）

開いたままの括弧を、内側から順に閉じる。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例２：括弧の種類と順序を保つ

入力（全文）／89 記号

```
[ ] { [ ] ( { ( [ [ { } ] ] ) } ) [ ( [ ( ( [
] { [ ( ) ( ) ] } ) ) ] [ { } ] ) { ( ( { [ (
) ( { } [ { [ [ [ [ ( ( ) ( ) [ [ ] ] ) ] ] ]
{ [ ] } ] } ] [ ( ) ] ) ] } ) [ [ ] ] )
```

正解：}] }

開いたままの括弧を、内側から順に閉じる。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例３：長い列から必要な括弧を取り出す

入力（全文）／ 63 記号

```
{ [ [ { ( { } ) } ] ] } [ ( { ( ( { ( { [
( [ [ ( { ( ) } ) [ [ ] ] ] { ( [ ] [ ] )
} ] [ { [ [ ] ] } ] ) ] } ) } ) ( ( ) ) )
```

正解： `} ) ]`

開いたままの括弧を、内側から順に閉じる。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

正解は、残ったスタックを逆順に閉じる列

- 実例 1 で開いたままなのは、外側から丸括弧、角括弧、丸括弧の順。内側から閉じる正解は「)]) 」。
- 入力を読み終えたときにスタックに残った括弧を、逆順に閉じれば最短の補完が得られる。
- 生成器も同じ方法で正解を計算するため、括弧の種類と順序が合う答えが一つに決まる。

[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

正解の求め方と出力例

括弧を正しく閉じるよう指示する

補完問題に添える英語の指示と日本語訳

Please complete the rest of the following Dyck sequences, making sure that the parentheses are closed properly.

以下の Dyck 列の残りを補完し、括弧が正しく閉じられるようにしてください。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

必要な閉じ括弧が、順序どおりにそろうか

- 補完の正しさは、必要な閉じ括弧が種類・順序・個数のすべてで合うかで決まる。列全体を正解と比較すれば、余分な括弧や途中の取り違えも分かる。
- 短い正解でも、入力に深い入れ子や対応済みの括弧対が多く含まれるため、開いたままの括弧を最後まで保持する必要がある。

[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

余分な括弧も、補完の誤りになる

実例 3 への既存の出力例： text-davinci-003 （ 2023 年公開）

正解： })]

予測：] })]

予測の先頭には、正解にない「] 」が一つある。必要な 3 記号を含んでいても、この出力では括弧列を正しく完成できない。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

生成方法とデータの構成

確率的な文法で括弧列を生成する

生成規則	確率
S → 開く括弧 i S 閉じる括弧 i	p/n （各種類）
S → S S	q
S → 空列	1 − p − q

S は生成する括弧列。 n=3 、 p=0.5 、 q=0.25 として、入れ子と連結を繰り返す。

[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

完成列を作り、補完すべき短い列を求める

- 文法から完成した括弧列を生成し、指定した長さの範囲に入る候補を採る。
- 完成列の後ろから切る位置を決め、入力を作る。切り落とす側には、対応済みの部分列が含まれる場合もある。
- 残した入力をスタックで読み直し、必要な閉じ括弧を求める。この生成例では、正解の長さを 1 〜 3 記号にする。

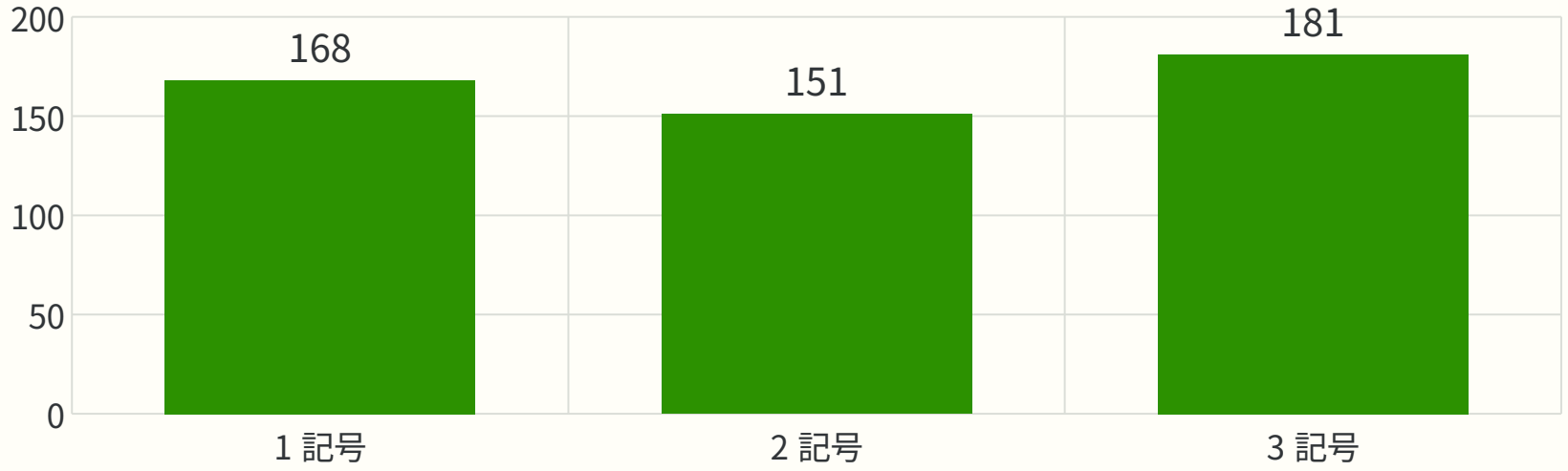
[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

短い列から、長い列への汎化を調べる

- 短い完成列（4 〜 50 記号）から作った問題を例として与え、より長い完成列（52 〜 100 記号）から作った問題で試す。
- 切り出し後の入力には、既に対応の終わった括弧対と、まだ閉じていない括弧が混在する。列が長くなっても、開いた順序をたどる規則は同じである。

[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

正解の長さは 1 〜 3 記号に分かれる



生成器の一設定で作った 500 問。補う閉じ括弧の記号数ごとの件数。

[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

公開された問題例から、入力と正解を読む

```
import json
with open("scenario_state.json", encoding="utf-8") as f:
    state = json.load(f)
example = state["request_states"][0]["instance"]
print(example["input"]["text"])
print(example["references"][0]["output"]["text"])
```

scenario_state.json は、公開された問題と既存出力の記録。最初の問題の入力と、生成器が計算した正解を表示する。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

研究での使われ方と入手

2019 年の研究は、各時点の次記号を予測する

- Suzgun らは 2019 年、外部メモリを持つ再帰型ニューラルネットが、入れ子構造を持つ Dyck 言語を学べるか調べた。
- 各時点で、それまでの列に続けられる記号の集合を予測する。全時点の予測が正しい列を成功と数えた。
- 次に許される記号を逐次選ぶ課題と、欠けた末尾を生成する課題では、出力と採点の単位が異なる。

[1] Suzgun et al. (2019). Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages. arXiv. https://arxiv.org/abs/1911.03329

Dyck-2 の研究では、外部メモリが役立つ

2019 年の方式	テスト成功率の平均
Vanilla RNN	0.00%
Vanilla LSTM	1.39%
Stack-RNN + Softmax	99.99%

10 回の実験。学習 5,000 列（2 〜 50 記号）、テスト 5,000 列（52 〜 100 記号）。

[1] Suzgun et al. (2019). Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages. arXiv. https://arxiv.org/abs/1911.03329

研究ごとに、括弧の種類と答え方が異なる

対象	データと出力
BIG-bench	Dyck-4 、固定 1,000 例、選択肢の得点で評価
生成器	Dyck-3 、条件を指定して括弧列を生成

BIG-bench は 1,000 問の固定データを公開する。補完問題の生成器では、括弧の種類や長さ、問題数を指定できる。

[1] BIG-bench. Dyck Language — 固定の括弧補完問題。
https://github.com/google/BIG-bench/tree/main/bigbench/benchmark_tasks/dyck_languages

[2] Stanford CRFM. DyckLanguageScenario — 括弧列の生成。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

生成器から、種類と長さを変えた問題を作る

- Stanford CRFM 研究チームの公開生成器は、括弧の種類数、問題数、長さ、乱数 seed を指定して補完問題を作る。外部コーパスのダウンロードは不要で、コードは Apache License 2.0 で提供される。
- 2019 年の著者実装は次記号の予測実験、 BIG-bench は固定の補完問題を公開する。調べたい構造や答え方に応じた資源がある。

[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

入れ子の規則を、長い列にも適用できるか

- Dyck 補完は、開いたままの括弧を内側から順に閉じる課題。括弧の種類・順序・個数が合う最短の正解を、スタックから一意に求められる。
- 確率的な文法で、入れ子や連結を含む問題を作れる。短い列で覚えた構造を、長い列にも適用できるかを調べる。
- 次記号を予測する研究、固定の補完問題、条件を変えられる生成器など、目的に応じた使い方がある。

[1] Stanford CRFM. DyckLanguageScenario — 括弧列の生成と正解計算。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

Appendix

1. 追加の実例
2. 研究と参考文献

実例 4 ：角括弧を一つ閉じる

入力（全文）／ 61 記号

```
[ ( ( [ [ ( ( { ( ) } ) ) { } ] { } { [ [
( ) ] ] } { [ [ [ [ ] ] ] ] } [ { { { { (
{ [ { ( ) } ] } [ ] ) } } } } ] ] ) )
```

正解：]　　予測：]

以降の予測は text-davinci-003 の 2023 年公開出力。この例では正解に一致している。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例 5 ：丸括弧を一つ閉じる

入力（全文）／ 77 記号

```
[ [ ] ] ( { } ) ( ( { { { { ( ( [ ( { }
) [ { } ] ] ) ) } } ( ) [ { [ [ ] ] ( {
[ ] } ) } ] [ { [ [ ] ] } ] [ { { } } ]
{ { ( [ ] { ( ) } ) } } ( ) } } )
```

正解：)　　予測：)

必要な括弧が一つで、予測も正解に一致している。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例 6 ：波括弧を一つ閉じる

入力（全文）／ 67 記号

```
( ( ) ) { [ ] { ( ( ( { [ ( ( ) ) ] { [ { ( )
} ] } [ [ ] ] ( ) [ [ { ( ( ( ( ( ) ) ) ) ) }
] [ ] { ( ) } { { } } ] ( ( ) ) } ) ) ) }
```

正解： }　　予測： }

必要な括弧が一つで、予測も正解に一致している。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例 7 ： 98 記号の入力を補う

入力（全文）／ 98 記号

```
( ) ( [ ] ) { } [ [ { { { ( ( ) ) } } [
( { { } } ) ] } ] ] ( [ ] ) [ ] ( [ [ [
{ { ( { [ { } ( ) ] } ) { [ [ ( ) ] ] }
} { { { [ ( { [ [ [ { { { } } } ] ] ] {
{ } } } ) { } ] } } } } ] ] { { } }
```

正解：])　　予測：)]

予測では二つの括弧の順序が入れ替わっている。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例 8 ：三つの括弧を逆順に閉じる

入力（全文）／ 53 記号

```
{ ( { { [ { ( [ ] [ ] ) } ] } } ( [
] [ [ [ ( ) ] ( [ [ ( ( ( [ [ ] { (
) } ] ) ) ) ] [ ( ) ] ] ) ] ] ) [
```

正解：]) }　　予測：]) })

予測には、正解より閉じ括弧が一つ多い。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例９：二つの括弧を閉じる

入力（全文）／ 70 記号

```
[ ( ( [ [ ] ] ( ) ) { ( [ [ { ( ) } ( ( [ ( { [
[ ( { [ ( [ ( [ ( [ ] ) ] ) ( ) [ [ { { ( [ ] )
} } ] ] ] ) ] } ) ] ] } ) ] ) ) ( ) ] ] ) }
```

正解：）]　　予測：）]

二種類の閉じ括弧が、正しい順序で予測されている。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例 10 ：同じ種類が離れて現れる

入力（全文）／91 記号

```
{ { } [ ] ( ) } [ ( [ [ { [ ] } ] ] ) ] [ ( {
( ( { { } } ) { } ) } ) [ { { { } } ( ( [ ( [
( { ( ) } ) ] ) ] [ ( ) ] ) ) { [ { } ] { [ ]
} { { } } } { } ( { } { [ ] } ( ) ) } ] ( [
```

正解：])]　　予測：])]

三つの閉じ括弧を、内側から逆順に閉じている。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例 11 ： 49 記号の入力を補う

入力（全文）／ 49 記号

```
[ ( { ( [ { [ ( { [ [ [ ( ) ] ] ]
} ) ] } ] ) { ( ( ( { } ( ) ( ) (
( [ ( ) ] ) ) ) ) [ ( ) ] ) }
```

正解： })]　　予測： })]

短めの入力にも、異なる種類の入れ子が含まれる。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例 12 ：複数の種類が重なる

入力（全文）／ 89 記号

```
[ { } ] [ ] ( { ( [ ] ) } [ ( { [ [ { [ ( [ [
] ] ) { ( ) } { [ [ ] ] } ] } ] ] } ) ] ( ) )
{ { [ ( { } ) ] } { } { [ ( ) ] } { } ( { { }
( ) ( ( ) ) } ) [ { [ ( { [ ] } ) { } ]
```

正解：}] }　　予測：}]]

予測の最後の括弧が、必要な種類と異なっている。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

実例 13 ：二つの閉じ括弧が必要

入力（全文）／ 74 記号

```
[ { ( { { { { } ( [ [ { } ] ] ) [ ] }
} } ( ( ( ) ) ) { [ ] } [ [ ( [ ] (
[ ( [ ] ) ] { { ( ( ) ) } } ) ) [ [ {
} ] ] ] [ ( ) ( ) ] ( ( ) ) ] } (
```

正解：）]　　予測：） ）]

予測には、正解にない丸括弧が一つ多い。

[1] Stanford CRFM (2023). Dyck Language — 公開問題と既存出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

参考文献

Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages

Mirac Suzgun, Sebastian Gehrmann, Yonatan Belinkov, Stuart M. Shieber (2019). arXiv:1911.03329.

外部メモリを持つ再帰型モデルを提案し、 Dyck 言語や回文などで長い系列への汎化を調べる。

https://arxiv.org/abs/1911.03329

Dyck Language

Mirac Suzgun et al. · n.d. · Google BIG-bench （ 2026-10-18 参照）

括弧列の欠けた末尾を選ぶ固定データと、生成文法や初期実験を掲載する。

https://github.com/google/BIG-bench/tree/main/bigbench/benchmark_tasks/dyck_languages

参考文献

Holistic Evaluation of Language Models

Percy Liang et al. (2023). Transactions on Machine Learning Research.

多面的な言語モデル評価の中で、括弧列を補う課題を扱う。

https://arxiv.org/abs/2211.09110

DyckLanguageScenario

Stanford CRFM (n.d.). 公開生成器。参照 2026-10-19 。

確率的文法による括弧列生成、入力の切り出し、スタックによる正解計算を実装する。

https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py